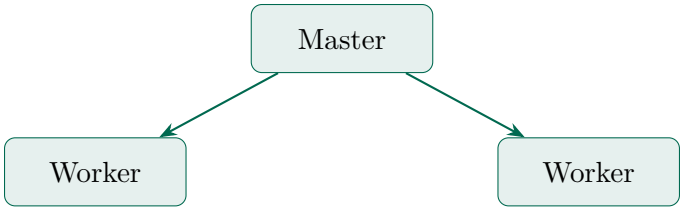
Universidad Pablo de Olavide

Máster Universitario en Ingeniería Informática

# Despliegue de un clúster Apache Spark

Computación de Altas Prestaciones — Práctica 1




**Autor:** Javier Gómez López

Curso 2026–2027 · Modalidad online

8 de noviembre de 2026

# Índice

**1. Introducción y objetivos** **2**

**2. Conceptos previos** **2**
2.1. ¿Qué es Apache Spark? . . . 2
2.2. Arquitectura de un clúster Spark . . . 2
2.3. Modo local frente a clúster . . . 3
2.4. Piezas de software necesarias . . . 4
2.5. El algoritmo K-Means . . . 4

**3. Instalación del software** **6**
3.1. Java (JDK) . . . 6
3.2. Scala . . . 6
3.3. Spark . . . 7
3.4. Servidor SSH . . . 7
3.5. Variables de entorno . . . 7

**4. Montaje del clúster** **7**
4.1. Intercambio de claves SSH . . . 7
4.2. Configuración de Spark . . . 8
4.3. Arranque del clúster . . . 9

**5. Explotación del clúster** **10**
5.1. Interfaz web del master . . . 10
5.2. Ejecución de K-Means en el clúster . . . 10
5.3. Comprobación del resultado . . . 11

**6. Problemas encontrados durante la realización de la práctica** **13**
6.1. El master no arranca por la versión de Java . . . 13
6.2. El worker no se registra en el master . . . 14

**7. Conclusiones** **15**

**Declaración de uso de inteligencia artificial** **15**

**Referencias** **16**

# 1. Introducción y objetivos

Cuando el volumen de datos crece hasta no caber (o no poder procesarse en un tiempo razonable) en una única máquina, la solución es *repartir el trabajo* entre varias. Esa es la idea central de la computación distribuida, y Apache Spark es hoy uno de los motores más utilizados para hacerlo.

En esta práctica se monta, desde cero, un pequeño clúster Spark de dos nodos (un **master** y un **worker**) sobre máquinas virtuales Linux, y se comprueba su funcionamiento ejecutando el algoritmo de *clustering* K-Means incluido en los ejemplos de Spark, siguiendo el enunciado de la práctica.

Los objetivos son:

1. Introducirse en el manejo de datos masivos (*Big Data*).
2. Comprender la arquitectura de un clúster Apache Spark y saber desplegarlo en modo *Standalone*.

# 2. Conceptos previos

## 2.1. ¿Qué es Apache Spark?

**Apache Spark**

Motor de procesamiento distribuido de propósito general. Permite escribir un programa como si se ejecutara en una sola máquina, y Spark se encarga de dividir los datos en trozos (*particiones*) y repartir el cómputo entre los nodos de un clúster.

Una de sus principales ventajas frente a su predecesor, Hadoop MapReduce, es que **puede** mantener datos en memoria para reutilizarlos entre operaciones, en lugar de leerlos y escribirlos en disco en cada paso. Conviene matizar esta idea:

- Spark no guarda en memoria automáticamente todos los resultados intermedios. Por defecto, si un conjunto de datos se usa varias veces, se vuelve a calcular cada vez desde su origen.
- Para reutilizarlo hay que pedirlo de forma explícita con `cache()` o `persist()`. Con `persist()` se elige el *nivel de almacenamiento*: sólo memoria, memoria y disco, sólo disco, etc. [1].
- Spark utiliza tanto memoria como disco: cuando los datos no caben en memoria puede volcarlos a disco (*spill*), y en las operaciones que redistribuyen datos entre nodos (*shuffle*) escribe los resultados intermedios en el disco local de los nodos [1].

Aun así, en algoritmos iterativos como K-Means, que recorren los mismos datos una y otra vez, poder conservarlos en memoria entre iteraciones supone una gran mejora de velocidad respecto a releerlos de disco en cada pasada.

La abstracción básica de Spark es el **RDD** (*Resilient Distributed Dataset*): una colección de elementos repartida entre los nodos, inmutable y capaz de reconstruirse si un nodo falla, porque Spark recuerda la secuencia de operaciones que la generó (su *linaje*) [1].

## 2.2. Arquitectura de un clúster Spark

Un clúster Spark tiene varios papeles bien diferenciados (Figura 1) [2]:

**Driver**
El proceso que ejecuta el programa principal (`main`) del usuario. Construye el plan de ejecución, lo divide en *tareas* y las envía a los ejecutores. Se lanza con `spark-submit`.

**Cluster Manager (Master)**
Lleva la cuenta de qué recursos (núcleos y memoria) hay disponibles en el clúster y los asigna a las aplicaciones. En esta práctica usamos el gestor propio de Spark, llamado *Standalone*; existen alternativas como YARN o Kubernetes.

**Worker**
Cada máquina que aporta recursos al clúster. Se registra en el master y, cuando éste se lo pide, lanza procesos *executor*.

**Executor**
Proceso JVM que vive en un worker, ejecuta las tareas que le manda el driver y, cuando se solicita con `cache()` o `persist()`, almacena (en memoria y/o disco) los datos que se quieren reutilizar.

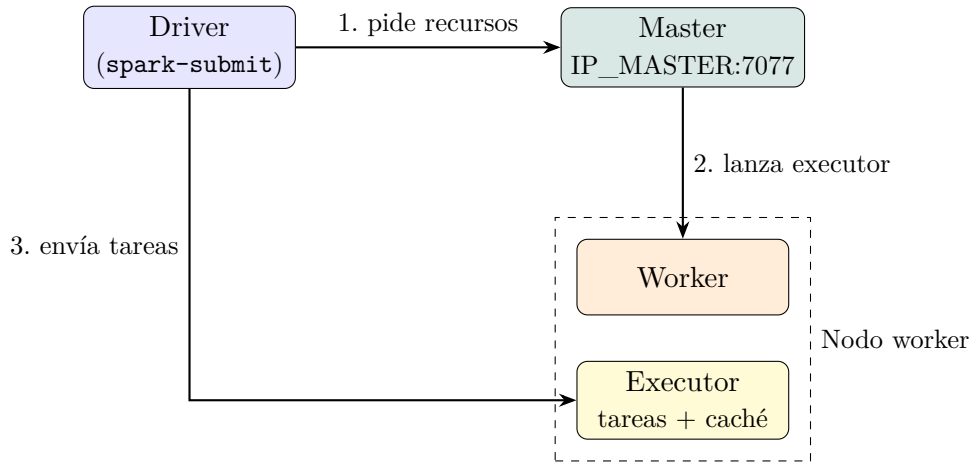



Figura 1: Arquitectura de un clúster Spark en modo Standalone.

El flujo es: el driver contacta con el master pidiendo recursos; el master ordena a los workers que arranquen executors; a partir de ahí el driver habla *directamente* con los executors para enviarles tareas y recoger resultados.

## 2.3. Modo local frente a clúster

Spark puede ejecutarse en **modo local** (`-master local[*]`), donde driver y ejecutores viven en un único proceso de una sola máquina. Es útil para desarrollar y probar. En **modo clúster** (`-master spark://IP:7077`) el trabajo se reparte realmente entre máquinas, que es lo que se persigue en esta práctica.

## 2.4. Piezas de software necesarias

| Componente | Por qué se necesita |
|---|---|
| JDK (Java) | Spark está escrito en Scala, que se compila a *bytecode* de la JVM. Sin Java, Spark no arranca. |
| Scala | Lenguaje nativo de Spark; necesario para compilar y ejecutar aplicaciones Scala propias. |
| Spark 3.2.1 | El motor en sí. Se usa la distribución precompilada para Hadoop 3.2. |
| OpenSSH | El master usa SSH para arrancar y parar los procesos remotos en los workers. |

## 2.5. El algoritmo K-Means

**K-Means**

Algoritmo de aprendizaje no supervisado que agrupa $n$ puntos en $k$ grupos (*clusters*), de forma que cada punto pertenece al grupo cuyo centro (*centroide*) está más cerca.

### Formulación

Se parte de un conjunto de $n$ puntos $x_1, \ldots, x_n \in \mathbb{R}^d$ y de un número de grupos $k$ fijado de antemano. K-Means busca $k$ centroides $\mu_1, \ldots, \mu_k$ y una asignación de cada punto a un grupo $C_1, \ldots, C_k$ que minimicen la suma de las distancias al cuadrado de cada punto a su centroide:

$$J = \sum_{j=1}^{k} \sum_{x_i \in C_j} \|x_i - \mu_j\|^2.$$

Esta cantidad es precisamente el *WSSSE* (*Within Set Sum of Squared Errors*) que imprime el ejemplo de Spark al terminar: cuanto menor es, más compactos son los grupos.

Encontrar el mínimo exacto de $J$ es un problema computacionalmente muy costoso, así que K-Means usa una estrategia iterativa que alterna dos pasos sencillos, cada uno de los cuales nunca empeora $J$:

**Algoritmo 1** K-Means

**Entrada:** puntos $x_1, \ldots, x_n$; número de grupos $k$; tolerancia $\varepsilon$
**Salida:** centroides $\mu_1, \ldots, \mu_k$ y asignación de cada punto a un grupo

1: Elegir $k$ centroides iniciales $\mu_1, \ldots, \mu_k$
2: **repetir**
3: **para** cada punto $x_i$ **hacer** ▷ Paso de asignación
4: $c_i \leftarrow \arg\min_j \|x_i - \mu_j\|^2$
5: **fin para**
6: **para** cada grupo $j = 1, \ldots, k$ **hacer** ▷ Paso de actualización
7: $\mu_j \leftarrow \dfrac{1}{|C_j|} \displaystyle\sum_{x_i \in C_j} x_i$
8: **fin para**
9: **hasta que** ningún centroide se desplaza más de $\varepsilon$ (o se alcanza un máximo de iteraciones)
10: **devolver** $\mu_1, \ldots, \mu_k$ y $c_1, \ldots, c_n$

- **Asignación:** con los centroides fijos, lo mejor que puede hacer cada punto es irse con el más cercano.

- **Actualización:** con los grupos fijos, el punto que minimiza la suma de distancias al cuadrado a los miembros de un grupo es su media. Por eso el centroide se recalcula como la media.
- **Convergencia:** como $J$ nunca aumenta y el número de asignaciones posibles es finito, el algoritmo siempre termina. Eso sí, llega a un *mínimo local*: el resultado depende de los centroides iniciales. Por eso es importante una buena inicialización.
- **Coste:** cada iteración calcula la distancia de cada punto a cada centroide, es decir, $O(n{\cdot}k{\cdot}d)$ operaciones. Con millones de puntos, repartir este cálculo entre varias máquinas es lo que justifica usar Spark.

**Ejemplo paso a paso**

Para verlo con números, tomamos seis puntos en el plano y $k = 2$:

$$A = (1,1), \quad B = (2,1), \quad C = (1,2), \quad D = (5,4), \quad E = (6,5), \quad F = (5,5).$$

A simple vista hay dos grupos: $\{A, B, C\}$ abajo a la izquierda y $\{D, E, F\}$ arriba a la derecha. Para que se vea cómo el algoritmo corrige una mala elección inicial, empezamos con dos centroides muy cerca uno del otro: $\mu_1 = A = (1,1)$ y $\mu_2 = B = (2,1)$.

**Iteración 1.** Se calcula la distancia al cuadrado de cada punto a los dos centroides (no hace falta la raíz cuadrada para saber cuál está más cerca):

| **Punto** | $\|x-\mu_1\|^2$ | $\|x-\mu_2\|^2$ | **Grupo** |
|---|---|---|---|
| $A = (1,1)$ | 0 | 1 | 1 |
| $B = (2,1)$ | 1 | 0 | 2 |
| $C = (1,2)$ | 1 | 2 | 1 |
| $D = (5,4)$ | 25 | 18 | 2 |
| $E = (6,5)$ | 41 | 32 | 2 |
| $F = (5,5)$ | 32 | 25 | 2 |

Se actualizan los centroides con la media de cada grupo:

$$\mu_1 = \tfrac{A+C}{2} = (1,\ 1.5), \qquad \mu_2 = \tfrac{B+D+E+F}{4} = \left(\tfrac{18}{4},\ \tfrac{15}{4}\right) = (4.5,\ 3.75).$$

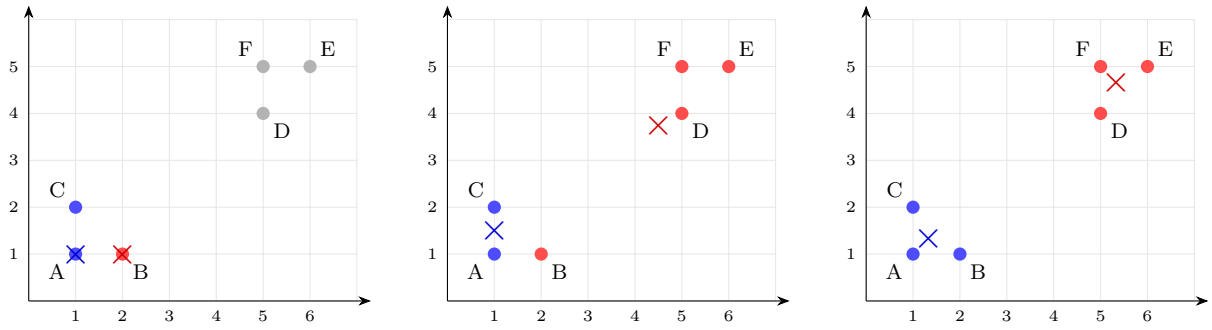
El centroide 2 ha dado un gran salto hacia el grupo de arriba, pero todavía arrastra al punto $B$.

**Iteración 2.** Con los nuevos centroides, $B$ queda ahora a distancia al cuadrado 1.25 de $\mu_1$ y 13.81 de $\mu_2$, así que cambia al grupo 1. El resto de puntos no cambia. Los grupos son ya $\{A, B, C\}$ y $\{D, E, F\}$, y los centroides pasan a ser:

$$\mu_1 = \tfrac{A+B+C}{3} = \left(\tfrac{4}{3},\ \tfrac{4}{3}\right) \approx (1.33,\ 1.33), \qquad \mu_2 = \tfrac{D+E+F}{3} = \left(\tfrac{16}{3},\ \tfrac{14}{3}\right) \approx (5.33,\ 4.67).$$

**Iteración 3.** Al repetir la asignación, ningún punto cambia de grupo, por lo que los centroides tampoco se mueven: el algoritmo ha **convergido**. El valor final es $J = \frac{4}{3} + \frac{4}{3} = \frac{8}{3} \approx 2.67$, frente a $J = 0.5 + 19.75 = 20.25$ tras la primera iteración.

La Figura 2 muestra la evolución gráficamente.




(a) Inicio (b) Tras la iteración 1 (c) Convergencia

Figura 2: Evolución de K-Means con $k = 2$. Las cruces son los centroides ($\mu_1$ en azul, $\mu_2$ en rojo) y el color de cada punto indica el grupo asignado. En (a) los puntos $D$, $E$, $F$ aún no tienen grupo; en (b) $B$ sigue asignado al grupo rojo; en (c) los grupos ya coinciden con los dos conjuntos naturales.

#### K-Means en Spark

El paso de asignación es *embarazosamente paralelo*: cada punto se procesa de forma independiente, así que Spark puede repartir los puntos entre los executors. Sólo la actualización requiere juntar resultados parciales: cada executor calcula, para cada grupo, la suma y el número de sus puntos asignados, y basta con combinar esas sumas parciales para obtener las medias. Como los mismos datos se recorren en cada iteración, éste es un caso típico en el que conviene mantenerlos en memoria con `cache()`. Por eso es un buen ejemplo para probar un clúster. La implementación de Spark MLlib, además, elige los centroides iniciales con una variante paralela de K-Means++ llamada K-Means‖ [3].

## 3. Instalación del software

Se parte de dos máquinas virtuales Linux creadas con UTM en un Mac, una para el master y otra para el worker. Todo este apartado se realiza **en las dos máquinas**, ya que Spark necesita las mismas versiones y las mismas rutas en todos los nodos. Ambas deben estar en la misma red para poder verse; la IP de cada una se obtiene con `ip a` o `hostname -I`.

### 3.1. Java (JDK)

```
sudo apt update
sudo apt upgrade
sudo apt install openjdk-11-jdk
```

`apt update` refresca la lista de paquetes disponibles, `apt upgrade` actualiza los ya instalados y `openjdk-11-jdk` instala el JDK 11. Puede comprobarse con `java -version`.

**Ojo**

Spark 3.2.1 sólo es compatible con Java 8 y 11 [4], por lo que se instala explícitamente el JDK 11 en lugar de `default-jdk`. Véase el apartado 6.1.

### 3.2. Scala

```
sudo apt-get install scala
```

### 3.3. Spark

Se descarga la versión 3.2.1 precompilada, se descomprime y se mueve a `/opt`, que es el directorio convencional en Linux para software instalado manualmente:

```
wget https://archive.apache.org/dist/spark/spark-3.2.1/spark-3.2.1-bin-hadoop3.2.tgz
tar -xvf spark-3.2.1-bin-hadoop3.2.tgz
sudo mv spark-3.2.1-bin-hadoop3.2 /opt/spark
sudo chown -R $USER /opt/spark
```

El último comando cambia el propietario de la carpeta a nuestro usuario. Es importante porque Spark escribe *logs* y ficheros de trabajo dentro de su directorio, y sin permisos de escritura fallaría al arrancar.

### 3.4. Servidor SSH

```
sudo apt install openssh-server
```

Los scripts de arranque del clúster se conectan por SSH a cada worker para lanzar allí el proceso correspondiente, por lo que cada nodo debe aceptar conexiones SSH.

### 3.5. Variables de entorno

Se añaden al final de `~/.bashrc` (fichero que se ejecuta al abrir cada terminal):

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

y se recargan con `source ~/.bashrc`.

- `JAVA_HOME` indica a Spark dónde está la instalación de Java.
- `SPARK_HOME` indica dónde está Spark.
- Añadir `bin` y `sbin` al `PATH` permite ejecutar `spark-submit`, `start-all.sh`, etc. desde cualquier directorio.

**Ojo**

La ruta de `JAVA_HOME` depende de la arquitectura de la máquina: en x86-64 es `java-11-openjdk-amd64` y en ARM (p. ej. máquinas virtuales en un Mac con Apple Silicon) es `java-11-openjdk-arm64`. Puede consultarse con `ls /usr/lib/jvm/`.

## 4. Montaje del clúster

### 4.1. Intercambio de claves SSH

Como se ha dicho, el master arranca los procesos de los workers por SSH. Si cada conexión pidiera contraseña, el arranque automático sería imposible. La solución es la **autenticación por clave pública**.

**Autenticación SSH por clave pública**

Se genera una pareja de claves: una **privada**, que nunca sale de la máquina, y una **pública**, que se copia en las máquinas a las que queremos acceder. Al conectar, el servidor

comprueba que el cliente posee la clave privada correspondiente sin que ésta viaje por la red, y permite el acceso sin pedir contraseña.

Desde el master:

```
ssh-keygen          # aceptar los valores por defecto
ssh-copy-id -i ~/.ssh/id_rsa.pub usuarioWorker@IP_WORKER
ssh-copy-id -i ~/.ssh/id_rsa.pub usuarioMaster@IP_MASTER
```

`ssh-keygen` crea `~/.ssh/id_rsa` (privada) e `id_rsa.pub` (pública). `ssh-copy-id` añade la clave pública al fichero `~/.ssh/authorized_keys` del destino. Se copia también al propio master porque los scripts de arranque se conectan a él mismo por SSH para lanzar su parte. Puede comprobarse con `ssh usuarioWorker@IP_WORKER`: debe entrar sin pedir contraseña.

Si el usuario del worker es distinto del del master, hay que indicárselo a SSH en el fichero `~/.ssh/config` del master (véase el apartado 6.2):

```
Host IP_WORKER
  User usuarioWorker
```

## 4.2. Configuración de Spark

Spark necesita saber dos cosas: *quién es el master* y *quiénes son los workers*. Ambas se configuran en `/opt/spark/conf`, en los dos nodos:

```
cd /opt/spark/conf/
nano workers
cp spark-env.sh.template spark-env.sh
nano spark-env.sh
```

**Fichero `workers`.** Lista, una por línea, las máquinas en las que se arrancará un worker:

```
# A Spark Worker will be started on each of the machines listed below.
# localhost
IP_WORKER
```

**Ojo**

El enunciado llama a este fichero `slaves`, pero en Spark 3.2.1 el script de arranque lee `conf/workers` [5]. Véase el apartado 6.2.

**Fichero `spark-env.sh`.** Variables que leen los scripts al arrancar cada proceso:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export SPARK_MASTER_HOST=IP_MASTER
export SPARK_WORKER_CORES=1
export SPARK_WORKER_MEMORY=1g
```

- `JAVA_HOME`: se repite aquí porque el worker se lanza por SSH en una shell que no lee `~/.bashrc` (apartado 6.2).
- `SPARK_MASTER_HOST`: IP en la que escucha el master. Si no se fija, podría escuchar sólo en `localhost` y el worker no podría conectarse.
- `SPARK_WORKER_CORES`: núcleos que cada worker *ofrece* al clúster.

- `SPARK_WORKER_MEMORY`: memoria que cada worker ofrece para sus executors.

Estos valores no tienen por qué coincidir con el hardware real: son los recursos que el worker pone a disposición del master para repartir. Aquí se limitan a 1 núcleo y 1 GB porque las máquinas virtuales tienen pocos recursos.

### 4.3. Arranque del clúster

Desde el master:

```
cd /opt/spark/sbin/
./start-all.sh
```

`start-all.sh` es un atajo que llama a dos scripts [5]:

1. `start-master.sh`: arranca el proceso master en la máquina local, que queda escuchando en el puerto **7077** (para aplicaciones y workers) y sirve una interfaz web en el puerto **8080**.
2. `start-workers.sh`: lee el fichero de workers y, por SSH, arranca un proceso worker en cada máquina listada, indicándole la dirección del master para que se registre.

Para detener el clúster se usa el script simétrico `stop-all.sh`.

Puede verificarse con el comando `jps` (lista los procesos Java en ejecución): en el master debe aparecer `Master` (Figura 3) y en el worker, `Worker` (Figura 4). En ambos casos aparece también `Jps`, que es el propio proceso del comando.

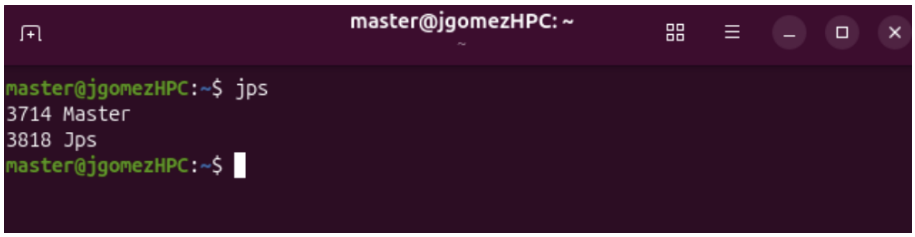



Figura 3: Salida de `jps` en el nodo master: proceso `Master` en ejecución.

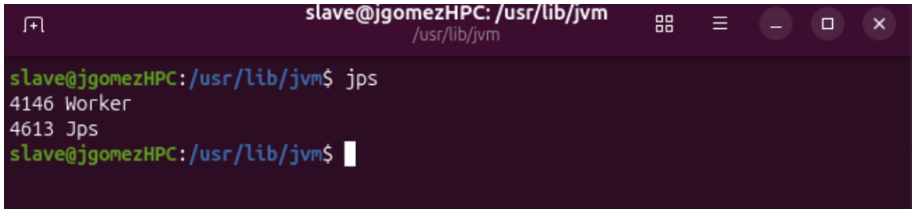



Figura 4: Salida de `jps` en el nodo worker: proceso `Worker` en ejecución.

También es visible desde el monitor del sistema del worker (Figura 5).

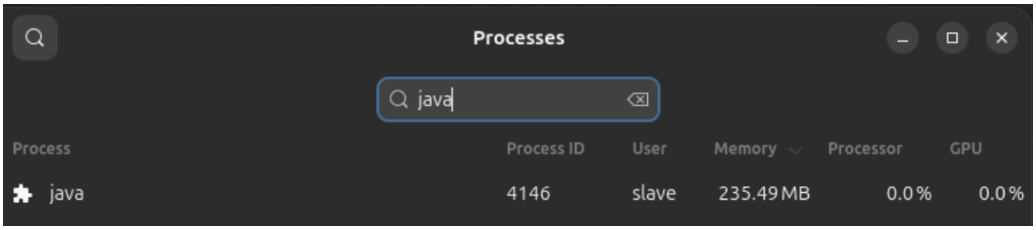



Figura 5: Proceso Java del worker visto desde el monitor del sistema.

# 5. Explotación del clúster

## 5.1. Interfaz web del master

Accediendo desde el navegador a `http://IP_MASTER:8080` se obtiene el panel de control del clúster (Figura 6). En él se puede ver:

- La URL del clúster, `spark://IP_MASTER:7077`, que es la dirección a la que se envían las aplicaciones.
- La tabla de **workers** registrados, con su identificador, dirección, estado (`ALIVE`) y recursos ofrecidos.
- Las aplicaciones en ejecución y las ya finalizadas.

Que el worker aparezca en esta tabla con estado `ALIVE` confirma que el montaje es correcto: se ha registrado en el master a través de la red.

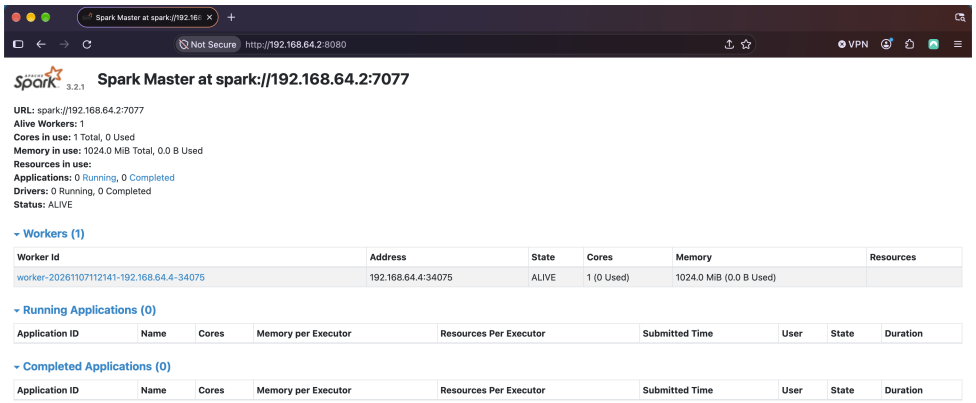



Figura 6: Interfaz web de Apache Spark en el master, con el worker registrado.

**Ojo**

La Figura 2 del enunciado muestra 2 workers y 2 núcleos en total, mientras que aquí aparecen 1 worker y 1 núcleo. La diferencia no es un error del despliegue: la configuración del enunciado (una sola IP en el fichero de workers y `SPARK_WORKER_CORES=1`) da exactamente el resultado de la Figura 6. La captura del enunciado corresponde a otro montaje: usa Spark 1.3.1, sus workers ofrecen 800 MB en lugar de 1 GB y ambos comparten IP y se distinguen sólo por el puerto, es decir, son dos instancias en una misma máquina. Probablemente se obtuvo con `SPARK_WORKER_INSTANCES=2`, variable que el enunciado no incluye. Sus 2 núcleos son la suma de 1 núcleo por worker, ya que `SPARK_WORKER_CORES` se aplica a cada worker.

## 5.2. Ejecución de K-Means en el clúster

Spark incluye una serie de aplicaciones de ejemplo empaquetadas en un `.jar`. Se lanza la de K-Means desde el master:

```
cd /opt/spark
./bin/spark-submit \
  --master spark://IP_MASTER:7077 \
  --class org.apache.spark.examples.mllib.KMeansExample \
  examples/jars/spark-examples_2.12-3.2.1.jar
```

- `spark-submit` es la herramienta estándar para enviar aplicaciones a Spark. Arranca el *driver* en la máquina desde la que se ejecuta [6].
- `-master` indica a qué clúster enviarla. Cambiando este valor por `local[*]` la misma aplicación se ejecutaría en modo local, sin tocar el código.
- `-class` indica la clase que contiene el método `main`.
- El último argumento es el `.jar` con el código compilado. El sufijo `_2.12` indica que se compiló con Scala 2.12.

Este ejemplo lee el fichero `data/mllib/kmeans_data.txt`, que contiene unos pocos puntos en 3 dimensiones, entrena un modelo K-Means con $k = 2$ y muestra el error del agrupamiento (*WSSSE*, suma de los cuadrados de las distancias de cada punto a su centroide).

**Ojo**

La ruta del fichero de datos es local. Funciona porque Spark está instalado en la misma ruta en ambos nodos y el fichero de ejemplo existe en los dos. En un clúster real se usaría un sistema de ficheros compartido como HDFS, que, según el enunciado, es el contenido de la segunda parte de la práctica.

Mientras la aplicación se ejecuta, aparece en la interfaz web con su nombre, núcleos asignados, memoria por nodo, usuario y tiempo de ejecución (Figura 7). Pulsando sobre el worker se ven los executors que está ejecutando para la aplicación (Figura 8).

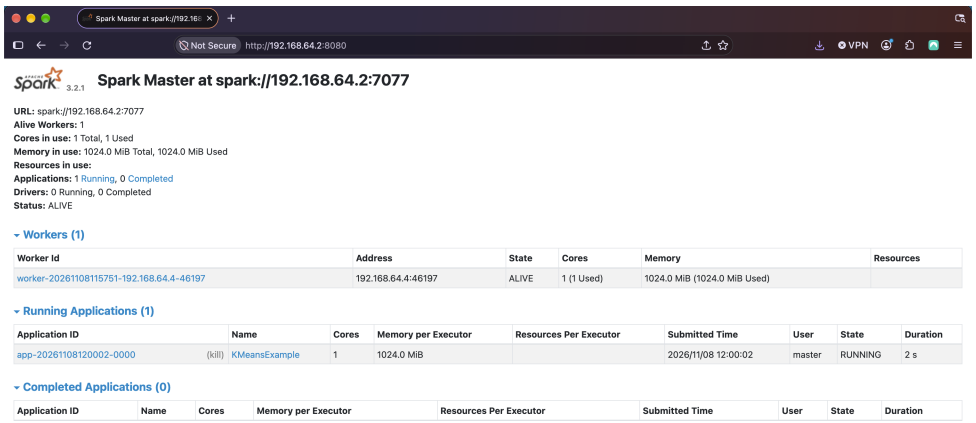



Figura 7: Aplicación K-Means ejecutándose sobre el clúster.

Al terminar, la aplicación pasa a la lista de *Completed Applications* (Figura 9).

## 5.3. Comprobación del resultado

Para comprobar que el clúster ha completado el cálculo basta con ver lo que imprime la aplicación. Como `spark-submit` se ejecuta en modo cliente, el *driver* corre en el master y su salida aparece en esa misma terminal, no en la interfaz web. Spark escribe sus mensajes de log por la salida de error, de modo que al descartarla con `2>/dev/null` queda únicamente el resultado del programa:

```
cd /opt/spark && ./bin/spark-submit \
  --master spark://192.168.64.2:7077 \
  --class org.apache.spark.examples.mllib.KMeansExample \
  examples/jars/spark-examples_2.12-3.2.1.jar 2>/dev/null
```

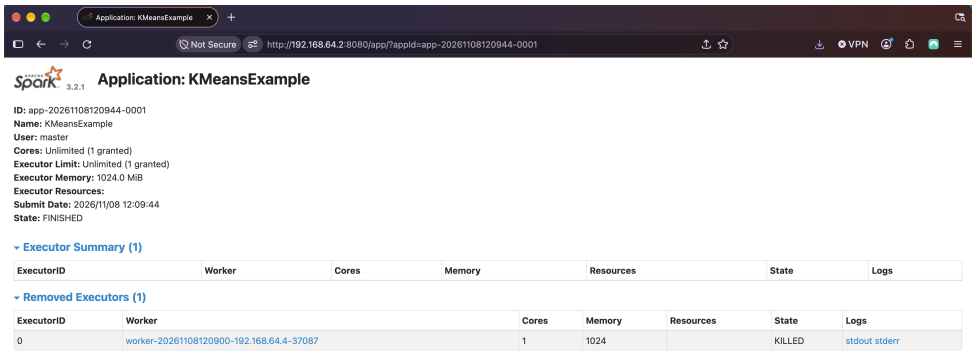



Figura 8: Detalle de ejecución en el worker.

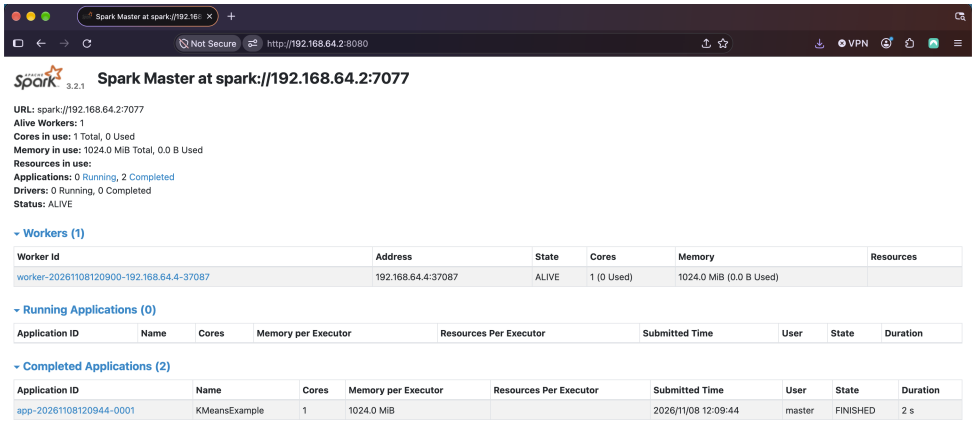



Figura 9: Aplicación ya finalizada en la interfaz web.

La línea `Within Set Sum of Squared Errors` (Figura 10) contiene el WSSSE del modelo entrenado. Que aparezca demuestra que el entrenamiento y el cálculo del error se han completado en el clúster.

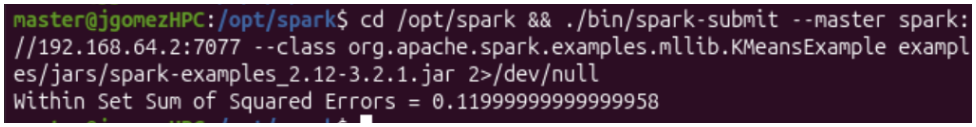



Figura 10: Salida de K-Means en la terminal del master, con el valor del WSSSE.

**Ojo**

Descartar la salida de error también oculta los fallos. Tras imprimir el WSSSE, el ejemplo guarda el modelo en una ruta local y lo vuelve a cargar, y esa carga termina con la excepción `UnsupportedOperationException: empty collection`: los ficheros los escriben los executors en el disco del worker, mientras que el driver los busca en el del master. No afecta al resultado ya impreso y es otra consecuencia de no disponer de un sistema de ficheros compartido como HDFS. Para repetir la ejecución hay que borrar antes, en ambos nodos, el directorio `/opt/spark/target/org/apache/spark/KMeansExample`.

# 6. Problemas encontrados durante la realización de la práctica

Seguir los pasos del enunciado al pie de la letra no fue suficiente para que el clúster funcionase. En este apartado se recogen los problemas que aparecieron, cómo se diagnosticaron y cómo se resolvieron. Todos tienen un origen común: el enunciado está pensado para un entorno algo distinto del utilizado (otra versión de Java, otros nombres de usuario y nombres de fichero de versiones anteriores de Spark).

## 6.1. El master no arranca por la versión de Java

**Síntoma.** Al ejecutar `start-all.sh`, el master se detiene nada más arrancar y el log muestra:

```
failed to launch: nice -n 0 /opt/spark/bin/spark-class
  org.apache.spark.deploy.master.Master --host 192.168.64.2 ...
Caused by: java.lang.IllegalStateException:
  java.lang.NoSuchMethodException: java.nio.DirectByteBuffer.<init>(long,int)
  at org.apache.spark.unsafe.Platform.<clinit>(Platform.java:113)
```

**Causa.** El enunciado instala Java con `sudo apt install default-jdk`, que en las distribuciones actuales instala una versión muy reciente (en este caso, Java 25). Spark 3.2.1 sólo es compatible con Java 8 y 11 [4]. Para gestionar memoria fuera del *heap*, Spark crea objetos `DirectByteBuffer` llamando por reflexión a un constructor interno de la JVM, `DirectByteBuffer(long, int)`. Ese constructor desapareció en versiones posteriores de Java, así que la llamada falla y el proceso termina.

**Solución.** Instalar el JDK 11 y hacer que Spark lo use:

```
sudo apt install openjdk-11-jdk
ls /usr/lib/jvm/          # comprobar la ruta exacta
```

A continuación se fija `JAVA_HOME` en `~/.bashrc` y en `spark-env.sh` (con la ruta `java-11-openjdk-amd64` o `java-11-openjdk-arm64`, según la arquitectura). Opcionalmente, puede elegirse Java 11 como versión predeterminada del sistema con `sudo update-alternatives -config java`.

Con Java 11 el log sigue mostrando un aviso `WARNING: An illegal reflective access operation has occurred`. Es el mismo acceso por reflexión que en Java 11 aún se permite, y puede ignorarse.

### 6.2. El worker no se registra en el master

**Síntoma.** Resuelto lo anterior, el master arranca correctamente (`jps` muestra `Master` y la interfaz web está disponible), pero la tabla de workers aparece vacía. En el worker, `jps` no muestra ningún proceso `Worker` y ni siquiera existe el directorio `/opt/spark/logs`.

**Diagnóstico.** La ausencia de logs en el worker es la pista clave: el proceso no ha llegado a lanzarse, así que el fallo no está en la red entre worker y master, sino en el paso previo, en el que el master entra por SSH en el worker para arrancarlo. Revisando ese paso se encontraron tres problemas, que fallaban a la vez:

1. **Nombre del fichero de workers.** El enunciado pide escribir la lista de workers en `conf/slaves`, que es el nombre usado en versiones anteriores de Spark. En Spark 3.2.1 el script `start-workers.sh` lee `conf/workers` [5]. Además, la IP escrita en el fichero tenía una errata (`192.168.4.4` en lugar de `192.168.64.4`), así que aunque se hubiera leído, el master habría intentado arrancar el worker en una máquina inexistente.

2. **Usuarios distintos en master y worker.** Los scripts se conectan con `ssh IP_WORKER`, es decir, con el mismo nombre de usuario que los ejecuta en el master (`master`). En el worker ese usuario no existe: el usuario es `slave`.

3. **Versión de Java en el worker.** `JAVA_HOME` sólo estaba definida en `~/.bashrc`. El master funcionaba porque se arranca desde una terminal interactiva, que sí lee ese fichero. Sin embargo, el worker se lanza por SSH en una shell *no interactiva*, y el `.bashrc` de Ubuntu termina inmediatamente en ese caso, sin llegar a las líneas añadidas. El worker habría arrancado con el Java por defecto del sistema (el 25) y habría fallado igual que el master en el apartado 6.1.

**Solución.** Cada problema se corrige por separado:

1. En el master, crear `conf/workers` con la IP correcta del worker:

```
echo 192.168.64.4 > /opt/spark/conf/workers
```

2. En el master, indicar a SSH qué usuario usar en el worker, en `~/.ssh/config`:

```
Host 192.168.64.4
  User slave
```

   Se comprueba con `ssh 192.168.64.4` (sin usuario), que debe entrar en el worker sin pedir contraseña.

3. En los dos nodos, añadir `JAVA_HOME` a `spark-env.sh`, que los scripts de Spark leen siempre, sea cual sea el tipo de shell:

```
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-arm64
```

Tras reiniciar el clúster con `stop-all.sh` y `start-all.sh`, el log del worker muestra `Successfully registered with master spark://192.168.64.2:7077` y el worker aparece en la interfaz web con estado `ALIVE`.

**Cómo diagnosticar un worker que no aparece**

Lo más rápido es comprobar en el worker si existe `/opt/spark/logs`. Si no existe, el proceso no llegó a lanzarse: hay que revisar el fichero `workers` y el SSH desde el master (`ssh IP_WORKER` debe entrar sin contraseña). Si existe, el log `spark-*-Worker-*.out` indica si arrancó y si logró conectar con el master. Si no conecta, el problema está en la red o en la dirección en la que escucha el master (`SPARK_MASTER_HOST`, puerto 7077).

# 7. Conclusiones

En esta práctica se ha desplegado un clúster Apache Spark en modo Standalone formado por un master y un worker, y se ha verificado su funcionamiento ejecutando un algoritmo de aprendizaje automático distribuido.

Las ideas principales que deja son:

- Un clúster Spark separa claramente quién *gestiona* los recursos (master), quién los *aporta* (workers) y quién *dirige* el cómputo (driver).
- El despliegue se apoya en herramientas estándar de Linux: SSH sin contraseña para el control remoto de los nodos y variables de entorno para la configuración.
- Todos los nodos deben tener el mismo software, en las mismas versiones y en las mismas rutas.
- El mismo programa puede ejecutarse en local o en un clúster cambiando sólo el parámetro `-master`: Spark oculta la complejidad de la distribución.
- El acceso a los datos es el siguiente cuello de botella: para que todos los nodos lean los mismos datos hace falta un sistema de ficheros distribuido como HDFS.

# Declaración de uso de inteligencia artificial

Para la elaboración de este documento se ha utilizado una herramienta de inteligencia artificial generativa (Claude, de Anthropic). La IA se ha empleado para redactar las explicaciones de los conceptos, estructurar la memoria y generar el código LaTeX y los diagramas. El contenido ha sido revisado por el autor, y el despliegue del clúster, las capturas de pantalla y el vídeo de funcionamiento corresponden a la realización práctica llevada a cabo por el autor.

La IA se ha utilizado únicamente como herramienta de apoyo. El alumno ha revisado, contrastado y validado todo el contenido, y asume la **plena responsabilidad** sobre el documento: sobre la exactitud técnica de las explicaciones, la corrección de los comandos y configuraciones descritos, la idoneidad de las fuentes citadas y el cumplimiento de la normativa académica de la Universidad Pablo de Olavide. Cualquier error u omisión que pueda contener es responsabilidad exclusiva del autor.

# Referencias

[1] The Apache Software Foundation. RDD programming guide — Spark 3.2.1 documentation. `https://archive.apache.org/dist/spark/docs/3.2.1/rdd-programming-guide.html`, 2022. Secciones *Shuffle operations* y *RDD Persistence*. Consultado el 26 de octubre de 2026.

[2] The Apache Software Foundation. Cluster mode overview — Spark 3.2.1 documentation. `https://archive.apache.org/dist/spark/docs/3.2.1/cluster-overview.html`, 2022. Consultado el 26 de octubre de 2026.

[3] The Apache Software Foundation. Clustering — RDD-based API — Spark 3.2.1 documentation. `https://archive.apache.org/dist/spark/docs/3.2.1/mllib-clustering.html`, 2022. Consultado el 26 de octubre de 2026.

[4] The Apache Software Foundation. Overview — Spark 3.2.1 documentation. `https://archive.apache.org/dist/spark/docs/3.2.1/index.html`, 2022. Consultado el 7 de noviembre de 2026.

[5] The Apache Software Foundation. Spark standalone mode — Spark 3.2.1 documentation. `https://archive.apache.org/dist/spark/docs/3.2.1/spark-standalone.html`, 2022. Consultado el 26 de octubre de 2026.

[6] The Apache Software Foundation. Submitting applications — Spark 3.2.1 documentation. `https://archive.apache.org/dist/spark/docs/3.2.1/submitting-applications.html`, 2022. Consultado el 26 de octubre de 2026.